Step 1 — The Security Reviewer on a Diff — Solution

> Back to the step · Hint 1 · Hint 2

Story: this builds the smallest piece first — a Security reviewer that scans a diff — proven on a deliberately planted vulnerability *and* a genuinely clean example, before anything else in the project exists. **If not:** a reviewer only ever tested on bad input will happily, falsely flag good code too, with no test ever catching it.

Setup files

`requirements.txt`: `openai`, `langchain-openai`, `langgraph`, `chromadb`, `python-dotenv`, `pydantic`. `.env.example`: `OPENAI_API_KEY=sk-your-key-here` / `LOG_LEVEL=INFO`. `.gitignore`: `.env`, `.venv/`, `__pycache__/`, `*.pyc`, `chroma_db/`. Every folder under `src/` gets an empty `__init__.py`. `LICENSE` is the MIT text.

Settings, errors, logging

```
# src/exceptions.py
class MissingConfigError(Exception):
    """A required setting is missing from the environment."""
    pass
```

```
# src/config.py
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError

@dataclass
class Config:
    openai_api_key: str
    log_level: str

def require_env(key):
    value = os.getenv(key)
```

```
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config():
    load_dotenv()
    return Config(
        openai_api_key=require_env("OPENAI_API_KEY"),
        log_level=os.getenv("LOG_LEVEL", "INFO"),
    )
```

```
# src/utils/logger.py
import logging
import os

def get_logger(name):
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler()
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        handler.setFormatter(logging.Formatter(log_format))
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

```
# src/models/llm.py
from langchain_openai import ChatOpenAI

def get_model(config):
    return ChatOpenAI(
        model="gpt-4o-mini", temperature=0, api_key=config.openai_api_key
    )
```

The finding shape and the diff parser

```
# src/schemas.py
# One review finding. severity is high/medium/low; category is which reviewer
# raised it; message is the human-readable issue.
```

```
from pydantic import BaseModel

class Finding(BaseModel):
    severity: str
    category: str
    message: str
```

```
# src/diff.py
# Parse a unified diff into the ADDED lines per file (like Doc08's chunking:
# only look at what actually changed), and categorize a file by extension.
def parse_diff(text):
    files = []
    current = None
    for line in text.splitlines():
        if line.startswith("+++ b/"):
            current = {"path": line[len("+++ b/"):], "added": []}
            files.append(current)
        elif line.startswith("+") and not line.startswith("+++"):
            if current is not None:
                current["added"].append(line[1:])
    return files

def categorize(path):
    if path.endswith((".md", ".rst", ".txt")):
        return "docs"
    if path.endswith((".yaml", ".yml", ".json", ".toml", ".ini", ".cfg")):
        return "config"
    return "code"
```

The Security reviewer

Story: a deterministic pattern scanner is the actual trustworthy core here — it looks only at the newly added lines for secret and unsafe code patterns, and returns a list of `Finding`s directly. You can layer an LLM pass on top for genuine judgment calls, but the scanner itself is what you can actually test reliably. **If not:** a purely prompted reviewer with no deterministic core is non-deterministic and hard to pin down confidently on a known-clean diff.

```
# src/agents/security_reviewer.py
# Scan the diff's ADDED lines for hardcoded secrets and unsafe patterns.
# Deterministic, so it's testable; a reviewer tested only on bad input would
# flag good code, so test a clean diff too.
import re

from src.schemas import Finding
```

```
SECRET_PATTERNS = [
    (r"AKIA[0-9A-Z]{16}", "hardcoded AWS access key"),
    (r"(?i)(password|passwd|secret|api[_-]?key)\s*=\s*['\"][^'\"]{6,}['\"]",
     "hardcoded secret/credential"),
]
UNSAFE_PATTERNS = [
    (r"\beval\(", "use of eval() on untrusted input"),
    (r"subprocess\.\w+\([^)]*shell\s*=\s*True", "subprocess with shell=True"),
    (r"pickle\.loads\(", "unsafe pickle.loads()"),
]

def run_security(files):
    findings = []
    for one_file in files:
        for line in one_file["added"]:
            for pattern, message in SECRET_PATTERNS:
                if re.search(pattern, line):
                    findings.append(Finding(
                        severity="high", category="security",
                        message=f"{one_file['path']}: {message}",
                    ))
            for pattern, message in UNSAFE_PATTERNS:
                if re.search(pattern, line):
                    findings.append(Finding(
                        severity="medium", category="security",
                        message=f"{one_file['path']}: {message}",
                    ))
    return findings
```

main.py (standalone)

```
# main.py  (Step 1 version)
from src.agents.security_reviewer import run_security
from src.diff import parse_diff

DIRTY = """--- a/app.py
+++ b/app.py
+api_key = "SUPERsecret12345"
+result = eval(user_input)
+x = 1
"""
CLEAN = """--- a/app.py
+++ b/app.py
+total = price * quantity
```

```
+return total
"""

def main():
    for label, diff in [("dirty", DIRTY), ("clean", CLEAN)]:
        findings = run_security(parse_diff(diff))
        print(f"\n=== {label} ===")
        if not findings:
            print("  no security findings")
        for finding in findings:
            print(f"  [{finding.severity}] {finding.message}")

if __name__ == "__main__":
    main()
```

The tests

```
# tests/unit/test_config.py
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "CODEGUARD_TEST_SETTING"

def test_missing_setting():
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")

def test_empty_setting():
    os.environ[TEST_SETTING] = ""
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 2 (empty setting)   -> {result}")

def test_config_loads():
```

```
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    print(f"case 3 (config loads)    -> {result}")

if __name__ == "__main__":
    test_missing_setting()
    test_empty_setting()
    test_config_loads()
```

```
# tests/unit/test_diff.py
# Run from the repo root: python -m tests.unit.test_diff
from src.diff import categorize, parse_diff

DIFF = """--- a/app.py
+++ b/app.py
 unchanged line
+added line one
+added line two
-removed line
"""

def test_parses_added_lines_only():
    files = parse_diff(DIFF)
    result = "OK"
    if len(files) != 1 or files[0]["path"] != "app.py":
        result = "WRONG"
    if files[0]["added"] != ["added line one", "added line two"]:
        result = "WRONG"
    print(f"case 1 (added lines only) -> {result}")

def test_categorize():
    result = "OK"
    if categorize("README.md") != "docs":
        result = "WRONG"
    if categorize("config.yaml") != "config":
        result = "WRONG"
    if categorize("app.py") != "code":
        result = "WRONG"
    print(f"case 2 (categorize paths) -> {result}")

if __name__ == "__main__":
    test_parses_added_lines_only()
    test_categorize()
```

```
# tests/unit/test_security.py
# Run from the repo root: python -m tests.unit.test_security
from src.agents.security_reviewer import run_security
from src.diff import parse_diff

DIRTY = """--- a/app.py
+++ b/app.py
+api_key = "SUPERsecret12345"
+result = eval(user_input)
"""
CLEAN = """--- a/app.py
+++ b/app.py
+total = price * quantity
"""

def test_flags_secret_and_eval():
    findings = run_security(parse_diff(DIRTY))
    messages = " ".join(f.message for f in findings)
    result = "OK"
    if "secret" not in messages or "eval" not in messages:
        result = "WRONG"
    print(f"case 1 (secret + eval flagged) -> {result}")

def test_clean_diff_no_findings():
    findings = run_security(parse_diff(CLEAN))
    result = "OK"
    if findings != []:
        result = "WRONG"
    print(f"case 2 (clean diff -> none)    -> {result}")

if __name__ == "__main__":
    test_flags_secret_and_eval()
    test_clean_diff_no_findings()
```

Expected output:

```
case 1 (missing setting) -> OK
case 2 (empty setting)   -> OK
case 3 (config loads)    -> OK
```

```
case 1 (added lines only) -> OK
case 2 (categorize paths) -> OK
```

```
case 1 (secret + eval flagged) -> OK
case 2 (clean diff -> none)    -> OK
```

The clean-diff case is the one that matters most: it proves the reviewer stays quiet on good code, not just that it's loud on bad code.